# 4장 파이썬

# 강의 목표

1. 파이썬 언어의 요소와 구조를 이해한다.
2. 파이썬에서 다루는 데이터 타입과 변수에 대해 이해한다.
3. 파이썬의 연산자, 조건문, 반복문의 종류를 알고 사용할 수 있다.
4. 파이썬의 컨테이너 객체를 이해하고 활용할 수 있다.
5. 파이썬에서 함수의 구조를 이해하고 활용할 수 있다.
6. 파이썬의 모듈을 이해하고 활용할 수 있다.
7. 파이썬의 파일 입출력을 이해하고 활용할 수 있다.

# 들어가기 전에



- 파이썬 언어 학습
  - 어렵지 않은 수준의 파이썬을 간단히 정리하는 수준
  - 파이썬 프로그램을 리눅스 환경에서 작성하고 실습
    - 앞으로 다룰 모든 학습은, 리눅스에서 파이썬 프로그램을 하므로 이 환경에 익숙해져함
- 예제 파이썬 코드는 ch04 디렉터리에 저장하고 실행

```
pi@pi:~ $ mkdir ch04
pi@pi:~ $ cd ch04
pi@pi:~/ch04 $
```

4

# 4.1 파이썬 언어

# 파이썬 언어



- 파이썬(Python)
  - 파이썬은 1991년 귀도 반 로섬(Guido van Rossum) 개발자의 의해 개발
- 특징
  - 플랫폼 독립적 언어
  - 인터프리터 방식으로 실행되는 언어
    - 인터프리터에 의해 바로 해석되어 실행, 컴파일하여 실행 파일 만들지 않음
  - 객체지향 언어
  - 동적 타입(dynamically typed)의 대화형 언어
  - 스크립트 언어
- 스크립트 언어의 특징
  - 인터프리터에 의해 실행
  - 단순하여 배우기 쉬움

# 파이썬 설치 및 확인



- python 프로그램 실행

```
pi@pi:~ $ mkdir ch04
pi@pi:~ $ cd ch04
pi@pi:~/ch04 $ python
Python 3.9.2 (default, Mar 12 2021, 04:06:34)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

  - python 프로그램이 실행되면 설치된 것임
  - 현재 파이썬 3.9.2가 설치되어 있음
  - [Ctrl+D]를 입력하면 종료

# 파이썬 실행 방식 2가지



- 2가지 방식
  - 대화식 실행 방식
    - 파이썬 코드를 한 개씩 실행시키는 방법
  - 파이썬 파일 단위로 실행하는 방식
    - 파이썬 프로그램이 담긴 파일을 실행시키는 방법
- 대화식 실행 방식

```
pi@pi:~/ch04 $ python
Python 3.9.2 (default, Mar 12 2021, 04:06:34)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> width = 10
>>> height = 20
>>> area = width * height
>>> print("면적은 ", area)
면적은 200
>>>
```

# 파이썬 파일 단위로 실행하는 방식



- 파이썬 파일 단위로 실행하는 방식
  - vi로 area.py 작성

```
width = 10
height = 20
area = width * height
print("면적은 ", area)
~
~
~
:wq!
```

  - 실행

```
pi@pi:~/ch04 $ python area.py
면적은 200
pi@pi:~/ch04 $
```

# 4.2 데이터 타입과 변수

# 식별자



- 식별자
  - 식별자(identifier)란 파이썬 프로그램의 변수, 상수(리터럴), 함수에 붙이는 이름
- 식별자의 규칙
  - 첫 번째 문자
    - 알파벳(A-Z, a-z), 언더스코어(_), $ 문자만 사용 가능
  - 두 번째 이상 문자
    - 알파벳, 언더스코어(_), 0-9, $ 문자 사용 가능
  - 대소문자 구분 – myHome과 myhome은 다른 변수

```
6variable          # (X) 숫자로 시작할 수 없음
student_ID         # (○)
_code              # (○) 맞지만 권하지 않음
if                 # (X) 예약어 if 사용 불가
%calc              # (X) % 사용 불가
bar, Bar;          # (○) bar와 Bar는 서로 다른 식별자임에 주의
```

# 주석문



- 주석문(comment)
  - 파이썬 프로그램 내에 설명이나 메모 목적으로 삽입한 텍스트
  - 프로그램 코드로 처리되지 않음
- 주석문의 2가지
  - 한 라인 주석문
    - # 으로 시작하여 그 줄 끝까지 주석 처리
  - 여러 라인 주석문
    - """와 """ 사이에 여러 라인을 주석 처리

```
"""
   이것은 처음 작성한 파이썬 프로그램입니다.
   작성자 : 황기태, 이재문
   작성일 : 4월 14일
"""
width = 10                  # 너비 10
height = 20                 # 폭 20
area = width * height       # 면적 계산
print("면적은 ", area)      # 면적 출력
```

```
면적은 200
```

# 문장 구분



- 파이썬 프로그램의 기본 단위는 문장(statement)
- 세미콜론(;)으로 문장과 문장을 구분
  - 한 줄에 한 문장만 있는 경우 세미콜론 생략 가능

```
i = i + 1                    # (○) 한 줄에 한 문장이므로 세미콜론 생략
j = j + 1;                   # (○)
k = k + 1; m = m + 1;        # (○) 한 줄에 여러 문장
n = n + 1 p = p + 1;         # (X) 첫 번째 문장(n = n + 1) 끝에 세미콜론(;) 필요
```

# 화면 출력



- print()를 이용한 화면 출력
  - 1개의 정보를 출력하는 사례

```
print(3)
print(2.5)
print(2+3)
print("hello")
```

```
3
2.5
5
hello
```

  - 2개 이상의 정보를 출력하려면 다음과 같이 콤마(',') 사용

```
print(3, 2.5) # 32.5 출력
print("파이썬 성적은 ", 90, "점입니다")
```

```
32.5
파이썬 성적은 90점입니다
```

- print()를 이용한 화면 출력
  - 변수에 저장된 값을 출력하는 사례

```
score = 90
text = "파이썬 성적은 "
print(text, score, "점입니다")
```

```
파이썬 성적은 90점입니다
```

# 화면 출력



- print() 함수의 end=" " 옵션 활용

```
print("2 + 3은")
print(2+3, "입니다.")
```

```
2 + 3은
5입니다.
```

```
print("2 + 3은", end=" ")
print(2+3, "입니다.")
```

```
2 +3은 5입니다.
```

```
print("C++", end="; ")
print("Java", end="; ")
print("Python", "; ")
```

```
C++; Java; Python;
```

# 화면 출력

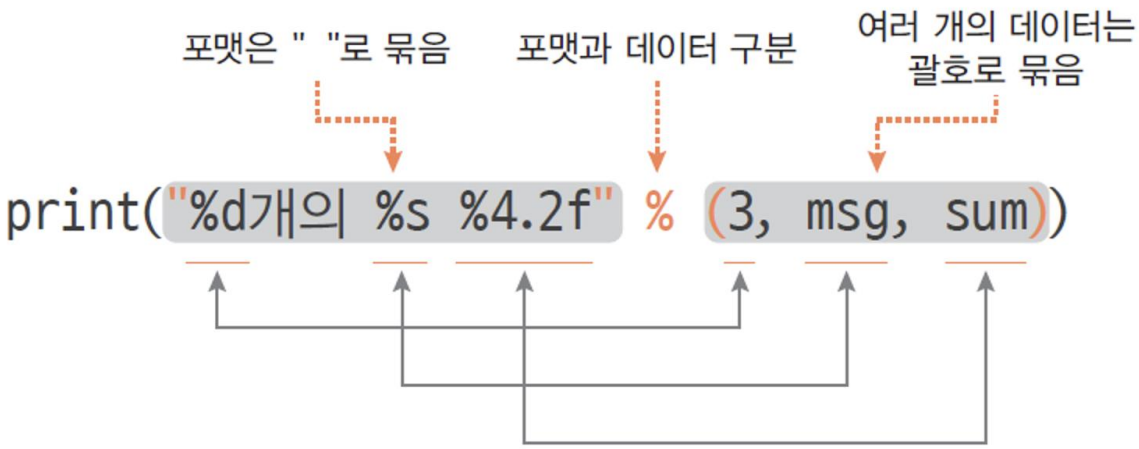


## 포맷 출력

포맷 문자 %s, %d, %f 활용

```
msg = "점수 합은"
sum = 98.7
print("%d개의 %s %4.2f" % (3, msg, sum))
```

3개의 점수 합은 98.70



# 키 입력



- input() 함수
  - 사용자로부터 키 입력
- input() 사례
  - 이름(문자열) 입력 사례

```
name = input("이름을 입력하세요>>") # 입력한 문자열이 name 변수에 저장
```

  - 정수 입력 사례

```
age = int(input("나이를 입력하세요")) # 입력을 정수로 바꾸어 age에 저장
```

  - 실수 입력 사례

```
weight= float(input("몸무게를 입력하세요")) # 입력을 실수로 바꾸어 weight에 저장
```

# 예제 4–1 이름과 점수를 입력 받아 평균을 출력하는 사례



이름과 과목 2개의 점수를 입력 받아 이름과 평균을 출력하라.

4-1.py

```
name = input("이름을 입력하세요>> ")
eng = int(input("영어 성적을 입력하세요>> "))
history = int(input("역사 성적을 입력하세요>> "))
print(name, "님은 성적 평균은 ", (eng+history)/2, "입니다.")
```

```
pi@pi:~/ch04 $ python 4-1.py
이름을 입력하세요>> 이재문
영어 성적을 입력하세요>> 80
역사 성적을 입력하세요>> 90
이재문 님은 성적 평균은 85.0입니다.
pi@pi:~/ch04 $
```

# 데이터 타입



- 데이터 타입(data type)
  - 파이썬 언어로 다룰 수 있는 데이터의 형태

- 데이터 타입의 종류
  - 숫자 타입 : 정수, 실수(예: 42, 3.14)
  - 논리 타입 : 참과 거짓을 표현하는 타입으로, True, False (true, false는 아니므로 주의할 것)
  - 문자열 타입 : '좋은 세상', "a", "365", "2+4"
  - 객체 레퍼런스 타입 : 객체를 가리킴. C 언어의 포인터와 유사
  - None : 값이 없음을 표시하는 특수 키워드, 다른 언어의 NULL, null과 유사

# 데이터 타입 및 변수 선언



- 변수는 파이썬 프로그램이 실행 중에 데이터를 저장하는 공간이며 반드시 이름을 가짐

- 변수='값'의 방식으로 매우 간단히 변수를 선언

```
age = 21      # 변수 age를 선언하고 21로 초기화
score = 10    # 정상적인 변수 선언
```

- 변수 사용

```
age = age + 1
score = 90

score = 66.8    # 실수도 저장 가능
score = "high"  # 문자열로 저장 가능
```

# 리터럴



- 변수가 데이터 저장 공간의 이름이라면, 리터럴(literal)은 데이터 값 그 자체

```
x = 10          # x는 변수, 10은 리터럴
```

- 문자열 리터럴
  - 단일 인용부호 사용: '황기태 '
  - 이중 인용부호 사용: "이재문 "

```
print("그녀는 ₩"누구세요₩"라고 했습니다.")
print('그녀는 "누구세요"라고 했습니다.')
```

# 리터럴



- 파이썬 리터럴의 종류와 예

| 구분 | | 특징 | 예 |
|---|---|---|---|
| 정수 | 8진수 | 0o로 시작(숫자 0과 문자 o) | n = 0o15 # 8진수, 10진수로 13 |
| | 10진수 | | n = 15 # 10진수 15 |
| | 16진수 | 0x로 시작(숫자 0과 문자 x) | n = 0x15 # 16진수 15, 10진수로 21 |
| 실수 | 소수형 | | height = 0.12345 |
| | 지수형 | | height = 12345E-4 # 1234*10-4 |
| 논리 | 참 | TRUE | condition = True # 참 |
| | 거짓 | FALSE | condition = False # 거짓 |
| 문자열 | | " "로 묶음 | hello = "안녕하세요" |
| | | ' '로 묶음 | name = 'kitae' |
| 기타 | None | 값이 없음을 뜻함 | ret = None # ret에 저장된 값이 없음 |

# 예제 4-2 리터럴 보기



파이썬의 다양한 리터럴 사용 예이다.

4-2.py

```
oct = 0o15 # 0o15는 8진수 15 -> 10진수로 13
hex = 0x15 # 0x14는 16진수 15 -> 10진수로 21
condition = True # 소문자 true로 하면 안 됨
print("8진수 0o15는 십진수로 %d" % oct)
print("16진수 0x15는 십진수로 %d" % hex)
print("condition은 " + str(condition))
print('문자열 : 단일인용부호로도 표현')
print("그녀는 ₩"누구세요₩"라고 했습니다.")
print('그녀는 "누구세요"라고 했습니다.')
```

```
pi@pi:~/ch04 $ python 4-2.py
8진수 0o15는 십진수로 13
16진수 0x15는 십진수로 21
condition은 True
문자열 : 단일인용부호로도 표현
그녀는 "누구세요"라고 했습니다.
그녀는 "누구세요"라고 했습니다.
pi@pi:~/ch04 $
```

# 상수



- 상수
  - 프로그램 실행 동안 고정된 값을 가지는 이름
- 파이썬에서 상수를 지원하지 않음
  - 개발자 스스로 변수에 초깃값을 저장한 이후 값을 바꾸지 않는 방식으로 활용
  - 일반적으로 대문자로만 상수명을 지음

```
PI = 3.14
GRAVITY = 9.8
UNIT = 1000
area = PI*3*3 # 반지름이 3인 원의 면적
```

# 데이터 형 변환



- 하나의 데이터 형의 데이터를 다른 데이터 형의 데이터로 변환
  - 문자열 "10" → 정수 10

```
total = int("10") + 20 # 문자열 "10"을 10으로 변환하고 20을 더해 30을 total에 저장
msg = "가격은 " + str(total) # total에 저장된 30을 "30"으로 변환하여, msg 변수에 "가격은 30"이 저장
```

오류

```
m = int("10.5") # 문자열 "10.5"를 정수로 바꿀 수 없어 오류 발생
n = int(10.5) # 실수 10.5를 10으로 바꿀 수 있어 오류 아님
```

# 데이터 형 변환



## 데이터 형 변환 함수들

| 함수명 | 내용 |
|---|---|
| int(value) | value를 정수로 변환하여 리턴, 변환 불가능하면 오류 발생하고 프로그램 종료 |
| float(value) | value를 실수로 변환하여 리턴, 변환 불가능하면 오류 발생하고 프로그램 종료 |
| str(value) | value를 문자열로 변환하여 리턴 |
| chr(value) | value를 문자열로 변환하여 리턴, value가 정수가 아니면 오류 발생하고 프로그램 종료 |
| bool(value) | value가 0이면 False, 그렇지 않으면 무조건 True로 하여 변환하여 리턴 |
| ord(value) | value가 1개의 문자로만 구성된 문자열이면, 문자에 대한 ASCII 코드 리턴 |

# 예제 4-3 형 변환 함수 활용



파이썬의 다양한 형 변환 함수를 사용하는 방법을 보인다.

4-3.py

```
ten = int("10")                    # ten = 10과 같은 결과
twenty = int(20.5)                 # twenty = 20과 같은 결과
text1 = str(30)                    # text1 = "30"과 같은 결과
text2 = str(40.5)                  # text2 = "40.5"
a = chr(65)                        # a = "A"와 같은 결과. 65는 문자 A에 대한 ASCII 코드
b = ord("A")                       # b = 65와 같은 결과. 문자 A의 ASCII 코드는 65
c = bool(0)                        # c = False와 같은 결과
d = bool("True")                   # d = True와 같은 결과
print("ten=%d, twenty=%d" % (ten, twenty))
print("text1=%s, text2=%s" % (text1, text2))
print("a=%s, b=%d, c=%d, d=%d" % (a, b, c, d))
```

```
pi@pi:~/ch04 $ python 4-3.py
ten=10, twenty=20
text1=30, text2=40.5
a=A, b=65, c=0, d=1
pi@pi:~/ch04 $
```

# 4.3 연산자와 식

# 연산자와 식



- 파이썬 연산과 연산자 종류

| 연산 | 연산자 |
|---|---|
| 산술 | +, -, *, /, //, % |
| 비교 | >, >=, <, <=, ==, != |
| 비트 | &, \|, ^, ~, <<, >> |

| 연산 | 연산자 |
|---|---|
| 대입 | =, +=, -=, *=, /=, //=, &=, \|=, ^=, <<=, >>= |
| 논리 | and, or, not |
| 조건 | if ... else |

# 산술연산



- 연산
  - 산술, 대입, 비교, 논리, 조건 연산 등
- 산술 연산
  - 더하기(+), 빼기(-), 곱하기(*), 실수 나누기(/), 정수 나누기(//), 나머지(%)의 6가지

```
x = 32;
total = 100 + x*2/4 –3;  // total은 113
```

```
x = 11/4;      # x는 3.75
y = 11//4;     # y는 2
```

```
x = 11%4;     # x는 3
y = 11%4.5      # y는 2.0
```

# 예제 4-4 산술 연산



다양한 산술 연산 예이다.

4-4.py

```
x=32
total = 100 + x*2/4 - 3          # total은 113
div = x / 10                     # div는 3.2
divint = x // 10                 # divint = 3
mod = x % 2                      # x를 2로 나눈 나머지, 0
print("x = %d" % x)
print("total = %d" % total)
print("div = %f" % div)
print("divint = %d" % divint)
print("mod = %d" % mod)
```

```
pi@pi:~/ch04 $ python 4-4.py
x = 32
total = 113
div = 3.200000
divint = 3
mod = 0
pi@pi:~/ch04 $
```

# 대입 연산



- 오른쪽 식의 결과를 왼쪽에 있는 변수에 대입

| 연산자 | 내용 |
|---|---|
| a = b | b값을 a에 대입 |
| a += b | a = a + b와 동일 |
| a -= b | a = a - b와 동일 |
| a *= b | a = a * b와 동일 |
| a /= b | a = a / b와 동일 |
| a //= b | a = a // b와 동일 |

| 연산자 | 내용 |
|---|---|
| a %= b | a = a % b와 동일 |
| a &= b | a = a & b와 동일 |
| a \|= b | a = a \| b와 동일 |
| a ^= b | a = a ^ b와 동일 |
| a <<= b | a = a << b와 동일 |
| a >>= b | a = a >> b와 동일 |

```
a = 1, b = 3
a = b     # a에 b의 값을 대입하여 a = 3, b = 3이 된다.
a += b      # a = a + b 의 연산이 이루어져, a = 6, b = 3이 된다.
```

# 예제 4-5 대입 연산



다양한 대입 연산 예이다.

4-5.py

```
x=3; y=3; z=3
print("x= %d y= %d z=%d" % (x, y, z))
x += 3 # x=x+3 -> x=6
y *= 3 # y=y*3 -> y=9
z %= 2 # z=z%2 -> z=1
print("x += 3; 실행 후, x= %d" % x)
print("y *= 3; 실행 후, y= %d" % y)
print("z %%= 2; 실행 후, z= %d" % z)
```

```
pi@pi:~/ch04 $ python 4-5.py
x= 3 y= 3 z=3
x += 3; 실행 후, x= 6
y *= 3; 실행 후, y= 9
z %= 2; 실행 후, z= 1
pi@pi:~/ch04 $
```

# 비교 연산



- >, >=, <, <=, ==, != 6가지 종류가 있으며 연산의 결과는 True/False

| 연산자 | 내용 |
|---|---|
| a < b | a가 b보다 작으면 True |
| a > b | a가 b보다 크면 True |
| a <= b | a가 b보다 작거나 같으면 True |

| 연산자 | 내용 |
|---|---|
| a >= b | a가 b보다 크거나 같으면 True |
| a == b | a가 b와 같으면 True |
| a != b | a가 b와 같지 않으면 True |

```
age = 25
result = (age > 20)     # age가 20보다 크므로 result는 True
```

```
name = "kitae"
res = (name == "kitae") # == 연산의 결과는 True, res = True
res = (name > "park") #name이 "park"보다 사전순으로 앞에 나오므로 res = False
```

# 예제 4-6 비교 연산



다양한 비교 연산 예이다.

4-6.py

```
x=13; y=7
print("x = %d, y = %d" % (x, y))
print("x == y : " + str(x == y))
print("x != y : " + str(x != y))
print("x >= y : " + str(x >= y))
print("x > y : " + str(x > y))
print("x <= y : " + str(x <= y))
print("x < y : " + str(x < y))
```

```
pi@pi:~/ch04 $ python 4-6.py
x = 13, y = 7
x == y : False
x != y : True
x >= y : True
x > y : True
x <= y : False
x < y : False
pi@pi:~/ch04 $
```

# 논리 연산



- and, or, not 3 가지 종류가 있으며 연산의 결과는 True/False

| 연산자 | 내용 |
|---|---|
| a and b | a, b 모두 True이면 True, 그렇지 않으면 False |
| a or b | a, b 중 하나라도 True이면 True, 둘 다 False이면 False |
| not a | a가 True이면 False, False이면 True |

```
score = 90
age = 20
res = ((score > 80) and (age < 25));   # res에 True가 대입된다.
```

# 예제 4-7 논리 연산



논리 연산의 다양한 예이다.

4-7.py

```
x=True; y=False
print("x = %s, y = %s" % (str(x), str(y)));
print("x and y : "+ str(x and y));
print("x or y : "+ str(x or y));
print("not x : " + str(not x));
print("");
print("(3>2) and (3<4) : " + str((3>2)and(3<4)));
print("(3==-2) or (-1<0) : " + str((3==2)or(-1<0)))
```

```
pi@pi:~/ch04 $ python 4-7.py
x = True, y = False
x and y : False
x or y : True
not x : False

(3>2) and (3<4) : True
(3==-2) or (-1<0) : True
pi@pi:~/ch04 $
```

# 조건 연산 및 문자열 연산



## 조건 연산

```
res = expT if( condition ) else expF
```

```
x=5; y=3
big = x if (x > y ) else y # ( x > y )가 True이므로 x 값 5가 big에 대입된다.
```

## 문자열 연산

- + 연산자가 있으며, 두 문자열을 연결하여 하나로 만듬

오류

```
"abc" + "de"     # "abcde"
"abc" + str(23) # "abc23"". "abc" + 23의 연산은 오류이다.
str(23) + "abc" # "23abc"
str(23) + "35"   # "2335"
23 + 35     # 58, 정수 더하기
```

# 예제 4-8 조건 연산



조건 연산 예이다.

4-8.py

```
a=3; b=5
print("a = %d, b = %d" % (a, b))
print("두수의 차이 : %d" % (a-b if a>b else b-a))
```

```
pi@pi:~/ch04 $ python 4-8.py
a = 3, b = 5
두수의 차이 : 2
pi@pi:~/ch04 $
```

# 예제 4-9 문자열 연산



문자열의 비교 연산 예이다.

4-9.py

```
print("abc" + str(23))
print(str(23) + "abc")
print(str(23) + "35")
print(str(23 + 35))
print(str(23 + 35) + "abc")
print("abc" + str(23 + 35))
name = "kitae"
print(name == "kitae")
print(name > "park")
```

```
pi@pi:~/ch04 $ python 4-9.py
abc23
23abc
2335
58
58abc
abc58
True
False
pi@pi:~/ch04 $
```

# 4.4 조건문

# 3가지 종류의 조건문



- 3가지 종류의 조건문
  - if, if-else, if-elif-else 종류가 있음
- if

```
if(조건식):
    … 실행문 … # 조건식이 True이면 실행
```

```
if(a > b):
    print("a가 b보다 큽니다.")
```

- if-else

```
if(조건식):
    … 실행문1 … # 조건식이 True인 경우 실행
else:
    … 실행문2 … # 조건식이 False인 경우 실행
```

```
if(a > 10):
    print("%d가 10보다 큽니다." % a)
else:
    print("%d가 10보다 작지 않습니다." % a)
```

# 3가지 종류의 조건문



- if-elif-else

```
if(조건식1):
    … 실행문1 … # 조건식1이 True인 경우
elif(조건식2):
    … 실행문2 … # 조건식 1이 False이고, 조건식2가 True인 경우 실행
……
else:
    … 실행문n … # 앞의 모든 조건이 False인 경우 실행
```

```
if(a > b):
    print("큰 수는 %d" % a);
elif(a < b):
    print("큰 수는 %d" % b);
else:
    print("%d와 %d는 같다." % (a, b));
```

# 블록과 들여쓰기



- 블록
  - 함수나, if, for, while 등에서 조건을 만족할 때 실행할 1개 이상의 문장들을 묶어 블록화
  - 대부분의 언어에서는 블록의 시작과 종료 기호를 별도로 사용
- 파이썬의 블록 시작
  - ':' 기호 사용
  - 블록 내 문장들은 동일한 크기의 들여쓰기
- 파이썬의 블록 종료
  - 들여쓰기의 종료

```
if(temperature > 30) :
    print("온도가 30이상.")
    print("경보 발송!")
    print("온도는 낮추도록 에어컨을 작동시켜라.")
else:
    print("온도가 30도 이하.")
```

블록의 시작

tab

들여 쓰기 일치

# 예제 4-10 if-elif-else와 들여쓰기 연습



if-elif-else 문을 활용하여 점수를 입력 받아 학점의 계산하여 출력하라.

4-10.py

```
grade = 0
score = input("점수를 입력하세요>>")
score = int(score) # 문자열을 숫자로 바꿈
if score >= 90 :       # score가 90 이상
   grade = "A"
elif score >= 80 :       # 80 이상 90 미만
   grade = "B"
elif score >= 70 :       # 70 이상 80 미만
   grade = "C"
elif score >= 60 :       # 60 이상 70 미만
   grade = "D"
else:          # 60 미만
   grade = "F"
print(str(score) + "는 " + grade + "입니다.")
```

```
pi@pi:~/ch04 $ python 4-10.py
점수를 입력하세요>>89
89는 B입니다.
pi@pi:~/ch04 $
```

# 4.5 반복문

# for



- 반복문
  - for, while이 있음, do-while은 없음
- for
  - 반복 회수를 아는 경우에 주로 사용

```
for 변수 in range(start, stop, step):
    반복될 문장1
    ...
    반복될 문장n
```

  - start
    - 시작 값. 생략하면 0
  - stop
    - 반복 중지 조건 값. 생략될 수 없음
  - step
    - 반복 시마다 증감 시키는 값. 생략하면 1

# for



- range() 함수의 사용 예

```
range(5)          # range(0,5,1)과 동일. 0, 1, 2, 3, 4를 순서대로 리턴
range(2, 5)       # range(2,5,1)과 동일. 2, 3, 4를 순서대로 리턴
range(2, 5, 2)    # 2, 4를 순서대로 리턴
range(5, 0, -1)   # 5, 4, 3, 2, 1을 순서대로 리턴

range(5, 2)       # range(5,2,1)과 동일. 아무 값도 리턴하지 않고 반복 종료
```

- 문자열과 for 반복문

```
for 변수 in 문자열:
   반복될 문장1
   ...
   반복될 문장n
```

```
for c in 'Falling in love with you':
   print(c, end=',')
```

```
F,a,l,l,i,n,g, ,i,n, ,l,o,v,e, ,w,i,t,h, ,y,o,u,
```

# 예제 4-11 range() 함수와 for 반목문



다양한 range ( ) 함수를 활용한 for 문의 예이다.

4-11.py

```
# 10에서 35까지 5씩 i 증가
for i in range(10, 36, 5):
    print(i, end=",")
print("")

# i를 0에서 5까지 1씩 증가
for i in range(6):
    print(10+i*5, end=",")
print("")

# i를 0에서 5까지 1씩 증가
for i in range(0, 6):
    print(10+i*5, end=",")
print("")

for i in range(35, 9, -5):
    print(i, end=",")
print("")
```

```
pi@pi:~/ch04 $ python 4-11.py
10,15,20,25,30,35,
10,15,20,25,30,35,
10,15,20,25,30,35,
35,30,25,20,15,10,
pi@pi:~/ch04 $
```

# 예제 4–12 문자열과 for 반복문



"Falling in love with you"에서 문자 " i"의 개수를 세는 코드를 작성하라.

4-12.py

```
count = 0
for c in "Falling in love with you" :
    if c == "i" :
        count += 1
print("i의 개수는 ", count)
```

```
pi@pi:~/ch04 $ python 4-12.py
i의 개수는 3
pi@pi:~/ch04 $
```

# while



- 조건식이 True인 동안 블록(문장1~문장N)을 반복 실행

```
while(조건식):
    반복될 문장1
    ...
    반복될 문장N
```

# 예제 4-13 while 문



while 문을 활용하여 정수를 입력받아 0에서 입력받은 정수까지의 합을 구하여 출력하는 프로그램을 작성하라..

4-13.py

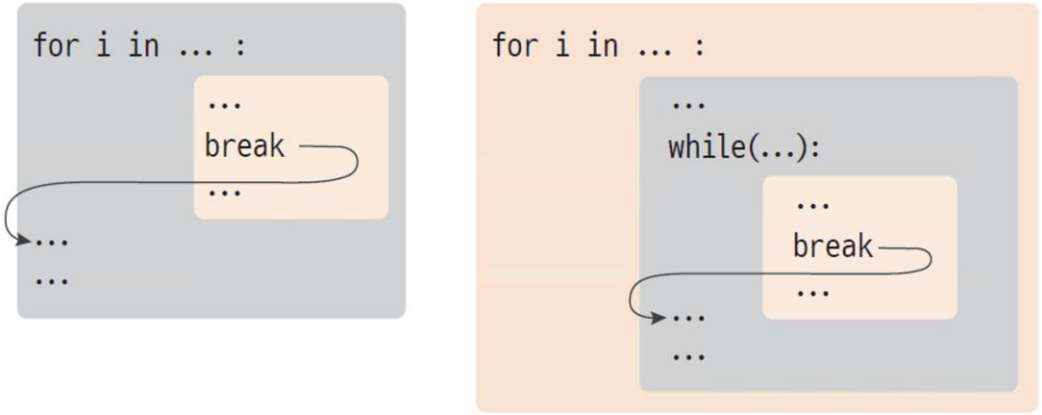
```
n = input("정수 입력>> ")
n = int(n) # 입력받은 문자열을 정수로 변환
i = 1
sum = 0
while i <= n :
    sum += i
    i += 1
print("1~%d까지의 합은 %d" % (n, sum))
```

```
pi@pi:~/ch04 $ python 4-13.py
정수를 입력하세요: 10
1~10까지의 합은 55입니다.
pi@pi:~/ch04 $
```

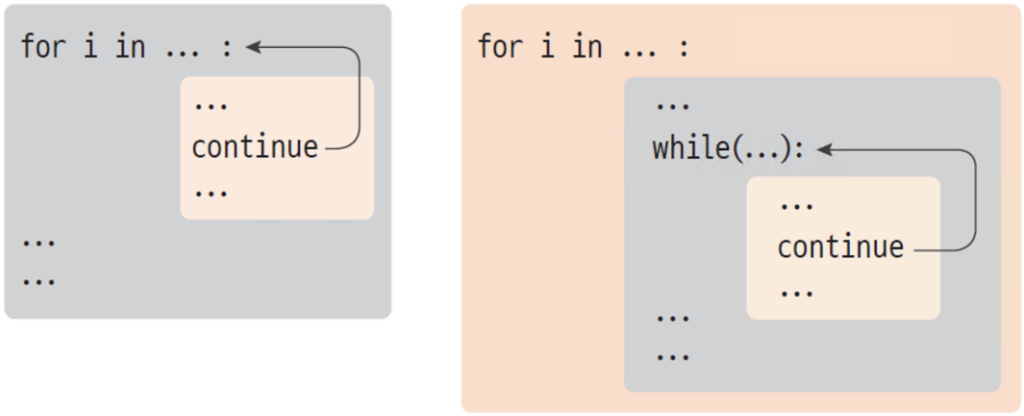
# 반복문 내의 break문 & continue문



- 반복문 내의 break문




- 반복문 내의 continue문



# 예제 4-14 break 문



break 문을 활용하여 반복문을 벗어나는 사례를 보인다. 1에서 얼마까지 더하면 3000보다 크면서 3000에 가장 가까운 합을 얻을 수 있는지 프로그램을 작성하여 찾아라.

4-14.py

```
i = 1
sum = 0
while True :
    sum += i
    if sum > 3000:
        break # 합이 3000을 넘어섰기 때문에 반복 종료
    i += 1
print("1~%d까지 합하면 %d입니다." % (i, sum))
```

```
pi@pi:~/ch04 $ python 4-14.py
1~77까지 합하면 3003입니다.
pi@pi:~/ch04 $
```

# 예제 4-15 continue 문



1에서 10 사이의 정수 중에서 3으로 나눈 나머지가 1인 정수들만 더한 합 구하라.

4-15.py

```
print("1~10사이 3으로 나눈 나머지가 1인 수만 더하기")
print("---------------------------------")
sum = 0
for i in range(1, 11): # i가 1에서 10까지 반복
    if (i % 3) != 1:
        continue
    print(i, end = " ")
    sum += i
print("의 합은 %d" % sum)
```

```
pi@pi:~/ch04 $ python 4-15.py
1~10사이 3으로 나눈 나머지가 1인 수만 더하기
---------------------------------
1 4 7 10 의 합은 22
pi@pi:~/ch04 $
```

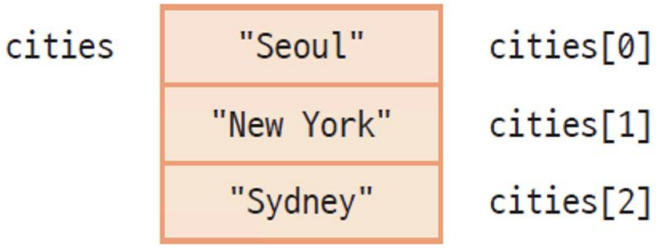
# 4.6 컨테이너 객체 다루기

# 리스트

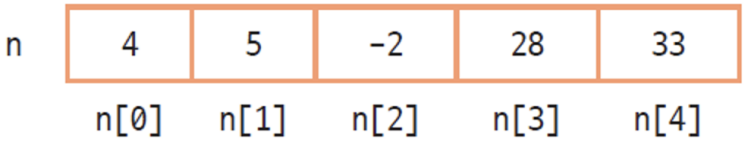


- 리스트(list)
  - 여러 개의 원소들을 연속적으로 저장하고 전체를 하나의 단위로 다루는 데이터 구조
  - 배열이라고도 함

```
cities = ["Seoul", "New York", "Sydney"]
```




```
n = [4, 5, -2, 28, 33]
```



# 리스트



- [] 연산자를 사용하여 리스트 생성

```
members = [] # 빈 리스트 생성
cities = ["Seoul", "New York"", "Syndey"]
week = ["월", "화", "수", "목", "금", "토", "일"]
plots = [-10, -20, 5, 10, 30]
```

```
yoyo = [1, "황기태", 2, "이재문", 3, 4, 5] # 서로 다른 타입의 데이터 저장 가능
```

- 리스트의 각 원소 접근

```
name = cities[0]      # name은 "Seoul"
cities[1] = "Gaineville"# "New York" 자리에 "Gaineville"이 저장됨
```

- 리스트의 원소 갯수

```
print( len( members ) )      # 0 출력
print( len( week ) )         # 7 출력
print( len( plots ) )        # 5 출력
```

# 리스트



## 리스트를 다루는 함수

| 함수 | 내용 |
|---|---|
| list.append(element) | list의 맨 마지막에 element 삽입 |
| list.pop() | list의 맨 마지막 원소 제거하고 그 값 리턴 |
| list.insert(index, element) | list의 index 번째 원소로 element 삽입 |
| list.remove(element) | list에서 첫 번째로 만나는 element 제거 |
| del list[index] | list에서 index 번째 원소 제거 |
| list.sort(reverse) | list의 원소들 정렬한다. reverse가 False이면 오름차순이고 True이면 내림차순이며 디폴트는 False |
| list.reverse() | list의 원소들을 역순으로 재정렬. list 내 원소 순서가 변경 |
| len(list) | list에 저장된 원소들의 개수를 리턴 |

# 예제 4-16 리스트 다루기



리스트를 다루는 다양한 예이다.

4-16.py

```
list = [1, 2, "jmlee", 3] # 리스트 만들기
print(list)
list.append(4)                          # 마지막 원소로 4 추가
print(list)

list.insert(3, "d")                     # 인덱스 3위치에 "d" 추가
print(list)

list[list.index("jmlee")] = 10          # "jmee"를 10으로 수정
list.remove("d")                        # "d" 원소 삭제
print(list)

list.sort(reverse = True)               # 내림차순으로 정렬
print(list)

print("원소의 개수 = %d" % len(list)) # 원소 개수
```

```
pi@pi:~/ch04 $ python 4-16.py
[1, 2, 'jmlee', 3]
[1, 2, 'jmlee', 3, 4]
[1, 2, 'jmlee', 'd', 3, 4]
[1, 2, 10, 3, 4]
[10, 4, 3, 2, 1]
원소의 개수 = 5
pi@pi:~/ch04 $
```

# 튜플



- 리스트와 유사하지만 다음 2가지가 다름
  - 튜플은 () 기호를 사용
  - 튜플은 원소의 값을 수정할 수 없음

```
cities = ("Seou", "New York", "Syndey") # 3개의 원소로 구성되는 튜플
my_home_town = cities[0] # my_home_town = "Seoul"
```

오류

오류

```
cities = ("Seoul", "New York", "Syndey")
del cities[0]      # 오류, 삭제 불가
cities[1] = "Gaineville"# 오류, 원소 값 수정 불가
```

```
cities = ("Seoul", "New York", "Syndey")
others = ("Busan", "Paris")
total = cities + others # total = ("Seoul", "New York", "Syndey", "Busan", "Paris")
```

# 딕셔너리



- <key, value>의 쌍을 하나의 원소로 저장하는 컨테이너
- 딕셔너리 생성: {}

```
members = {}   # 빈 딕셔너리
week = {"월":1, "화":2, "수":3, "목":4, "금":5, "토":6, "일":0}
nation = {"Korea":"Seoul", "Japan":"Tokyo", "France":"Paris"}
```

- 딕셔너리 원소 접근

```
dict = {}     # 빈 딕셔너리 dict 생성
dict[key] = value  # 한 쌍의 새로운 <key, value> 원소를 딕셔너리에 저장
val = dict[key]  # 저장되어 있는 <key, value>에서 value를 읽어 val에 저장
```

# 딕셔너리



- 딕셔너리를 다루는 함수

| 함수 | 내용 |
|---|---|
| dict.keys() | 딕셔너리에 저장된 모든 키들을 저장한 dict_keys 객체 리턴 |
| dict.values() | 딕셔너리에 저장된 모든 value들을 저장한 dict_values 객체 리턴 |
| dict.items() | 딕셔너리에 저장된 (키, 값)에 대한 dict_items 객체 리턴 |
| dict.clear() | 딕셔너리의 모든 원소 삭제 |
| dict.get(key) | key로 저장된 원소가 있으면 value 리턴, 없으면 None리턴 |
| del dict[key] | dict[key]를 제거 |

# 예제 4-17 딕셔너리 다루기



딕셔너리에서 최고 점수 또는 최저 점수를 찾는 프로그램이다.

4-17.py

```
dict = {"jmlee": 88, "kitae": 98, "ihjung":99}

cmd = input("high 또는 low를 입력하세요>> ")

names = list(dict.keys( ))
scores = list(dict.values( ))
if cmd == "high":
    high = 0
    for i in range(len(names)):
        high = scores[i] if high < scores[i] else high
    print("최고 점수는 %d입니다." % high)
if cmd == "low":
    low = 100
    for i in range(len(names)):
        low = scores[i] if low > scores[i] else low
    print("최저 점수는 %d입니다." % low)

value = dict.get("gdhong")
print(value)
```

```
pi@pi:~/ch04 $ python 4-17.py
high 또는 low를 입력하세요>> low
최저 점수는 88입니다
None
pi@pi:~/ch04 $
```

# 컨테이너와 for 문



- 컨테이너의 각 원소에 대하여 for 문을 적용 가능

```
for 변수 in 컨테이너:
    반복될 문장1
    ...
    반복될 문장n
```

# 예제 4-18 컨테이너와 for 반복문



딕셔너리와 배열에서 최고 점수 또는 최저 점수를 찾는 프로그램이다.

4-18.py

```
students = {"jmlee": [88, 97, 50], "kitae": [90, 80], "ihjung": [99, 70, 60]}

cmd = input("high 또는 low를 입력하세요>> ")

if cmd == "high":
    high = 0
    for name in students.keys( ):
        scores = students[name]
        for score in scores:
            high = score if high < score else high
    print("최고 점수는 %d입니다." % high)
else:
    low = 100
    for name in students.keys( ):
        scores = students[name]
        for score in scores:
            low = score if low > score else low
    print("최저 점수는 %d입니다." % low)
```

```
pi@pi:~/ch04 $ python 4-18.py
high 또는 low를 입력하세요>> high
최고 점수는 99입니다.
pi@pi:~/ch04 $
```

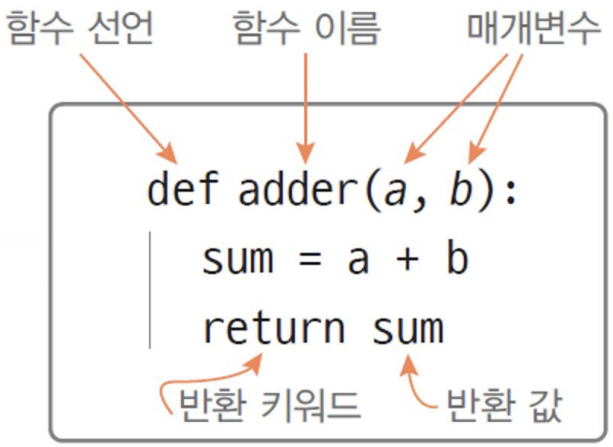
# 4.7 함수

# 함수의 구성



- 함수(function)는 데이터를 전달받아 처리한 후 결과를 돌려주는 코드 블록
- 함수의 구성

```
def 함수이름(arg1, agr2, ..., argn):
    ... 프로그램 코드 ...
    결과를 돌려주는 return 문
```




- def
  - 함수의 선언을 표시하는 키워드
  - def 문의 마지막에 콜론(':')을 사용하여 함수의 시작임을 알려야 함
- 함수이름
  - 개발자가 정하는 것으로 함수의 목적에 맞게 이름을 정함
- arg1, agr2, ..., argn
  - 함수를 호출하는 곳으로부터 값을 전달받는 매개변수
  - 여러 개가 있을 경우 콤마(,)로 분리. 값을 받지 않는 경우 매개변수 필요 없음
- 프로그램 코드
  - 함수의 작업을 실행하는 파이썬 코드. 프로그램 코드는 들여쓰기를 해야 함
- return 문
  - 함수의 실행을 종료하고 호출한 곳으로 되돌아가면서 함수의 실행 결과를 반환하는 문

# 함수 호출

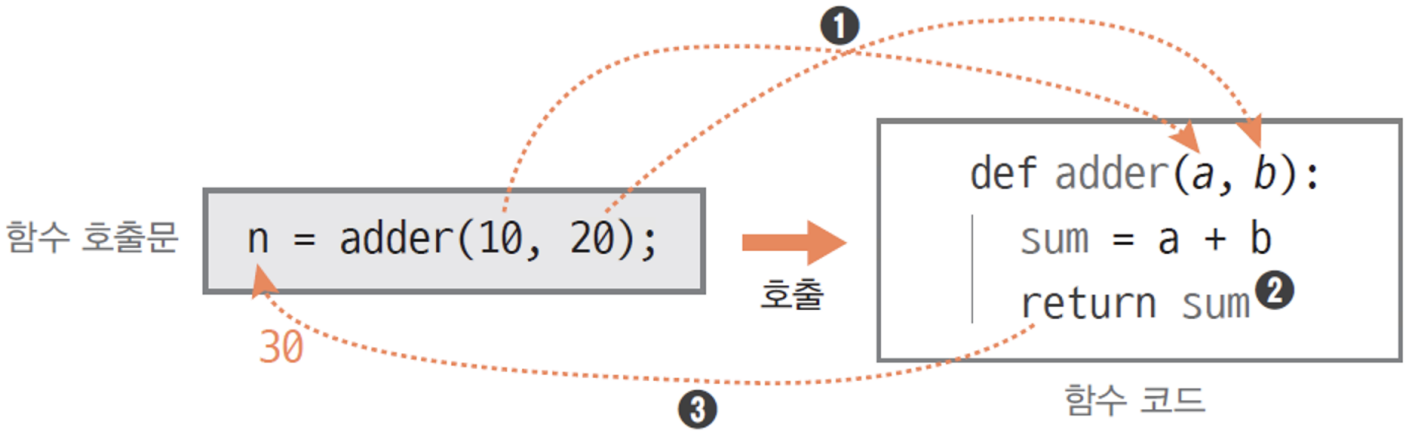


- 함수 호출(function call)
  - 함수에게 일을 지시하고 결과를 받는 것
- 방식

1) 변수이름 = 함수이름(arg1, arg2, ..., argn)
   - 함수 리턴 값 변수에 저장

2) 함수이름(arg1, arg2, ..., argn)
   - 함수로부터 리턴 값을 받지 않음

```
n = adder(10, 20) # adder() 함수 호출
```



# 매개 변수의 디폴트 값



- 매개변수에 디폴트 값이란?
  - 함수를 호출할 때 매개 변수에 값을 넘기는 것을 생략하면, 미리 정해진 값이 전달되는 방식
  - 함수를 선언할 때 디폴트 값 지정

```
def adder(a, b=20):
    sum = a + b
    return sum
```

```
sum = adder(10)       # adder(10, 20)과 동일, 매개변수 b에 디폴트 값 20 전달
sum = adder(10, 30)   # adder(10, 30)과 동일
```

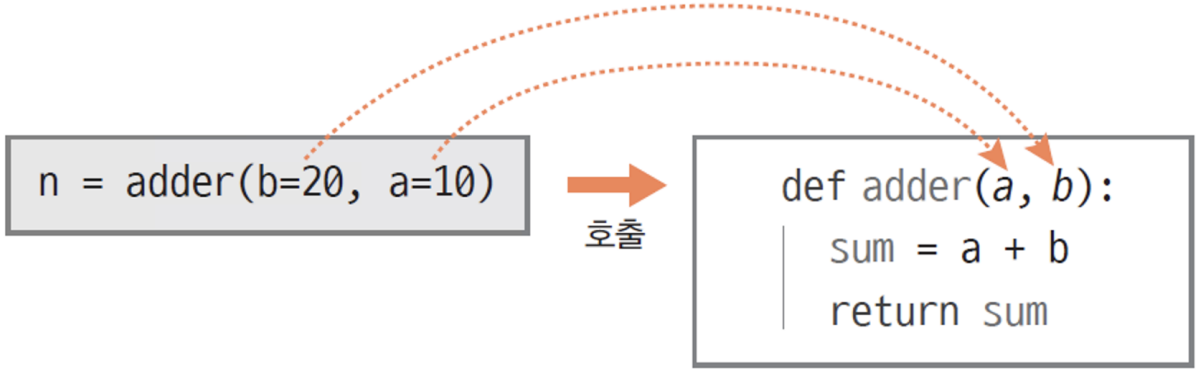
# 명명된 매개변수(named parameter)



- 명명된 매개변수
  - 함수 호출 문에 '매개변수이름=값'을 이용하여 매개변수에 값 전달

```
n = adder(a=20, b=30)
```

```
n = adder(b=30, a=20)
```



# 예제 4-19 함수 만들기



구구단을 출력하는 함수 gugudan ( )을 작성하고, 사용자로부터 정수를 입력받아 이 함수를 호출하여 구구단을 출력하는 프로그램을 작성하라.

4-19.py

```
def gugudan(n):
    for i in range(1,10):
        print("%d x %d = %d" % (n, i, n*i))

a = input("구구단 몇단을 원하세요>> ")
n = int(a)
gugudan(n)
```

```
pi@pi:~/ch04 $ python 4-19.py
구구단 몇단을 원하세요>> 6
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54
pi@pi:~/ch04 $
```

# 전역변수와 지역변수



- 전역 변수
  - 함수 밖에 선언되는 변수
  - 프로그램이 실행을 시작할 때 생겨나서 프로그램 종료까지 생존
  - 모든 함수에서 사용 가능
- 지역 변수
  - 함수 내에서 선언되는 변수
  - 함수가 시작되면 생겨나서 함수가 종료되면 사라짐
  - 선언된 함수 내에서만 사용 가능
- 전역변수와 지역변수 선언

```
x = 10              # 전역 변수
def f():
   y = 50           # 지역 변수
   return x + y

t = f()                                 # 함수 f() 호출
print('전역 변수 x = ', x)              # 10 출력
print('함수 f()의 리턴값은 ', t)       # 60 출력
```

# 함수 내에서 전역 변수에 쓰기



- 함수 내에서 전역 변수에 쓰기
  - 전역 변수에 값을 쓰는 경우,
  - 반드시 global 키워드를 사용하여 전역 변수임을 명확히 하여야 함

```
x = 10          # 전역 변수
def f():
    global x                    # 함수 내에서 등장하는 x는 전역 변수임을 선언
    x = x + 10                  # 전역 변수 x에 대한 계산. x는 20이 됨
    y = 50                      # 지역 변수
    return x + y + 10           # 80 리턴

t = f() # 함수 f() 호출
print('전역 변수 x = ', x)          # 20 출력
print('함수 f()의 리턴값은 ', t)    # 80 출력
```

# 전역 변수 사용시 주의



- global 키워드를 사용하지 않은 경우 아래와 같은 오류 발생

오류

```
x = 10              # 전역 변수
def f( ):
   # global x       # 주석 처리함, x가 전역 변수임을 알리지 않음
   x = x + 10       # x는 지역 변수인데 미 초기화로 오류가 발생
   y = 50           # 지역 변수
   return x + y     # 80 리턴
t = f( ) # 함수 f( ) 호출
print("전역 변수 x = ", x)              # 10 출력
print("함수 f( )의 리턴 값은 ", t)      # 80 출력
```

```
Traceback (most recent call last):
File "/home/pi/ch04/a.py", line 8, in <module>
t = f( ) # 함수 f( ) 호출
File "/home/pi/ch04/a.py", line 4, in f
x = x + 10 # x는 지역 변수인데 미 초기화로 오류가 발생
UnboundLocalError: local variable 'x' referenced before assignment
```

# 4.8 모듈

# import 키워드로 모듈 임포트



- 모듈
  - 모듈(module)이란 하나의 파이썬 프로그램 파일(.py 파일)

- 모듈이 임포트 되는 과정
  - 하나의 모듈에서 다른 모듈을 사용하기 위함

```
# 모듈 a, a.py 파일에 저장
name = '황기태' # 모듈 a의 전역변수
def printName():
    print('이름 = %s' % name)
printName()
```

```
pi@pi:~/ch04 $ python a.py
이름 = 황기태
pi@pi:~/ch04 $
```

# import 키워드로 모듈 임포트



- 모듈이 임포트되는 과정

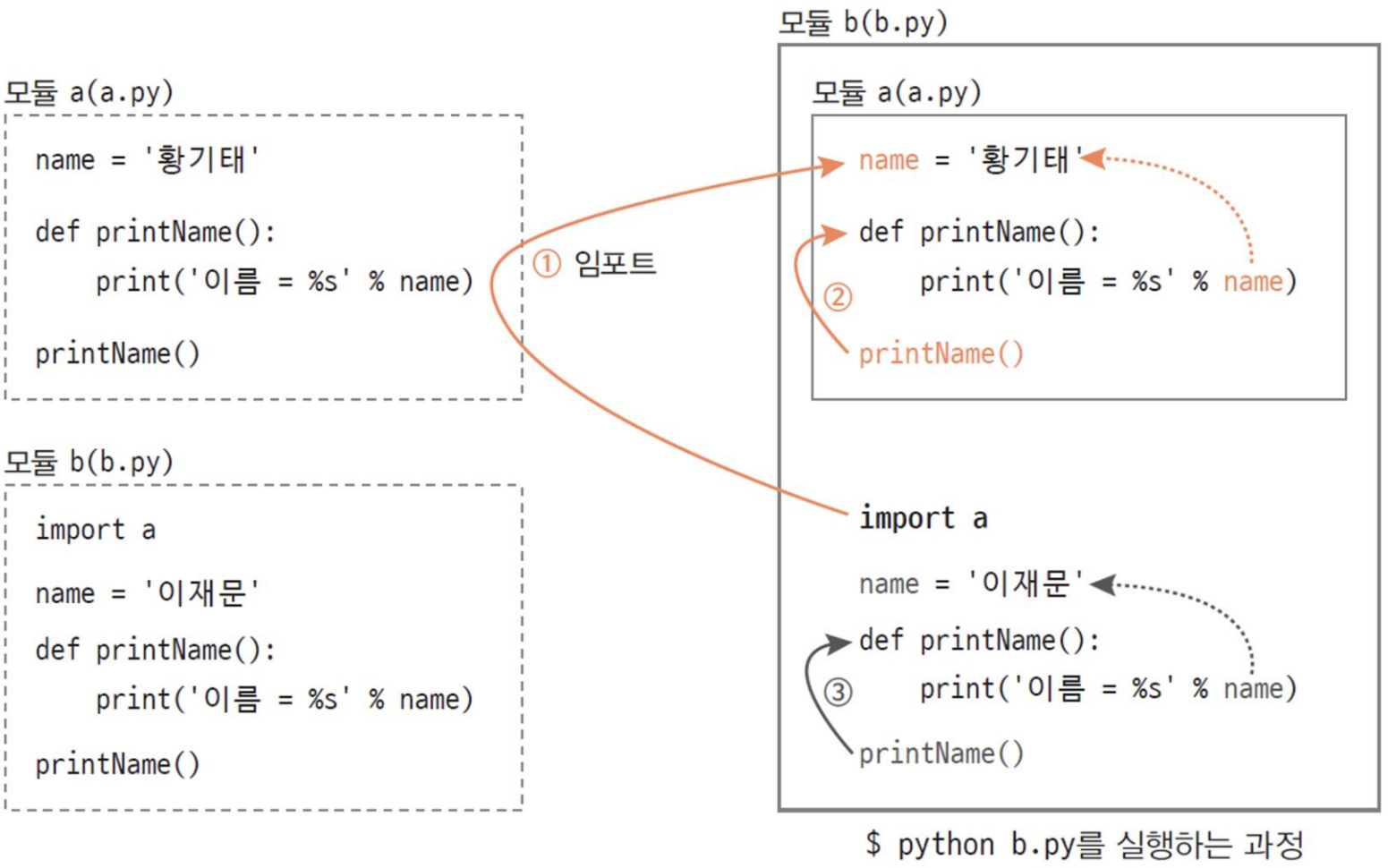
```
# 모듈 b. b.py 파일에 저장
import a # 임포트할 때 모듈 이름만 사용
name = '이재문' # 모듈 b의 전역변수
def printName():
   print('이름 = %s' % name)
printName()
```

```
pi@pi:~/ch04 $ python b.py
이름 = 황기태
이름 = 이재문
pi@pi:~/ch04 $
```

# import 키워드로 모듈 임포트



- 임포트된 모듈 활용



# import 키워드로 모듈 임포트



- 임포트된 모듈 활용
  - 접근방법
    - 모듈이름.전역변수
    - 모듈이름.함수

```
# 모듈 c. c.py에 저장
import a
name = '이재문'
def printName():
    print('모듈 c의 이름 = %s' % name) # 모듈 c의 name, '이재문' 출력

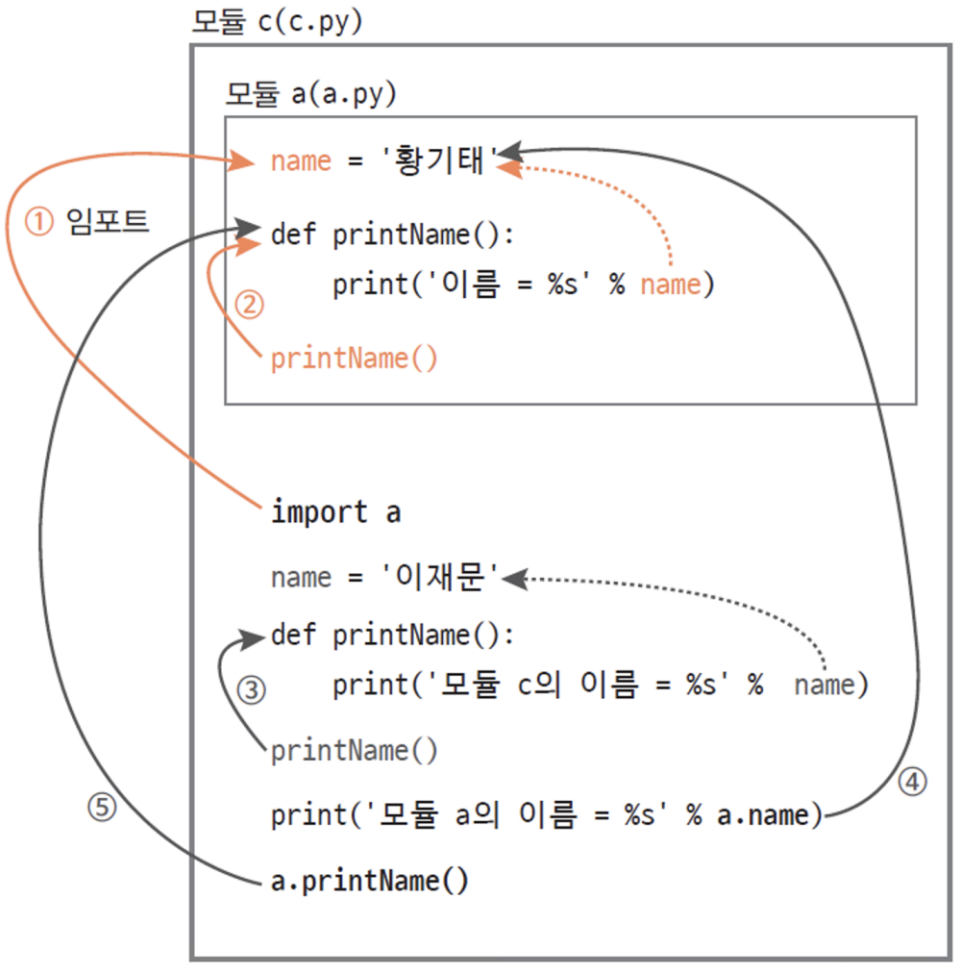
printName() # 모듈 c의 printName() 함수 실행
print('모듈 a의 이름 = %s' % a.name) # 모듈 a의 name, '황기태' 출력
a.printName() # 모듈 a의 printName() 함수 실행
```

```
pi@pi:~/ch04 $ python c.py
이름 = 황기태
모듈 c의 이름 = 이재문
모듈 a의 이름 = 황기태
이름 = 황기태
pi@pi:~/ch04 $
```

# import 키워드로 모듈 임포트



- 모듈이 임포트되는 과정




$ python c.py를 실행하는 과정

# 'from 모듈이름 import *'등으로 모듈 임포트



- 제약사항1
  - 임포트 될 모듈(a)과 임포트하는 모듈(b 또는 c)에서 동일한 식별자(전역변수 이름, 함수이름)가 있는 경우 임포트될 모듈의 식별자는 무시됨
- 제약사항2
  - 'from 모듈이름 import *'인 경우 임포트될 모듈에서 _로 시작하는 식별자는 임포트하는 모듈에서 액세스할 수 없음

```
# 모듈 d. d.py 파일에 저장
from a import name, printName
def printName():
   print('성함 = %s' % name) # name은 a모듈의 name임
printName()  # printName()함수는 이 모듈의 printName()임
```

```
pi@pi:~/ch04 $ python d.py
이름 = 황기태
성함 = 황기태
pi@pi:~/ch04 $
```

# __name__ 전역변수 활용



- __name__ 변수의 특징
  - 전역 변수임
  - 모듈이 단독으로 실행될 때, __name__ 변수에 '__main__'문자열 저장
  - 모듈이 임포트되어 실행될 때, __name__ 변수에 '모듈이름' 문자열 저장

```
# 모듈 a. a.py 파일에 저장
name = '황기태' # 모듈 a의 전역변수
def printName():
    print('이름 = %s' % name)
if __name__ == '__main__': # 모듈 a가 단독으로 실행될 때 이 조건문은 True
    printName()
```

```
pi@pi:~/ch04 $ python a.py
이름 = 황기태
pi@pi:~/ch04 $
```

# 예제 4-20 임포트 사례



다음과 같이 작성된 mymodule.py를 실행하였을 때 예상되는 출력과 mymodule을 임포트한 4-20.py를 실행할 때 예상되는 출력은 각각 무엇인가?

mymodule.py

```
delta = 10 # 전역변수. 모듈의 프로퍼티

def plus(value):
    global delta # 전역변수 delta
    delta += value
    return delta

if __name__ == '__main__' :
    print("mymodule 테스트 %d" % plus(100))

if __name__ == 'mymodule' :
    print("mymodule imported...₩n delta is initialized to 10₩n")
```

```
pi@pi:~/ch04 $ python mymodule.py
mymodule 테스트 110
pi@pi:~/ch04 $
```

4-20.py

```
import mymodule

ret = mymodule.plus(10)
print(ret)

ret = mymodule.plus(10)
print(ret)
```

```
pi@pi:~/ch04 $ python 4-20.py
mymodule imported...
delta is initialized to 10
20
30
pi@pi:~/ch04 $
```

# 파일처리



- 파일에 데이터를 쓰거나 읽기
- 파일에 저장된 데이터 종류
  - 바이너리 데이터
  - 텍스트 데이터
    - 이 책에서는 텍스트 데이터가 저장된 파일만 다룸
- 파일 처리 관련 주요 내장함수

| 내장함수 | 내용 |
|---|---|
| file = open(name, mode) | name 파일을 mode의 값에 따라 생성 혹은 열기. file 객체를 리턴하며, 추후 입출력은 file 객체 활용 |
| str = file.read() | file 객체가 가리키는 파일의 모든 데이터를 읽어 하나의 문자열(str)로 리턴 |
| file.write(str) | str의 문자열을 file 객체가 가리키는 파일에 저장 |
| lines = file.readlines() | file 객체가 가리키는 파일의 모든 데이터를 라인별로 읽어 문자열 배열(lines)로 리턴, 각 문자열(라인) 끝에는 '\n' 포함 |
| file.writelines([s1,s2,.., sn]) | file 객체가 가리키는 파일에 s1, s2, sn의 문자열을 순차적으로 저장. s1, s2, sn의 끝에는 '\n'이 포함되어 있어야 함 |
| file.close() | file 객체가 가리키는 파일을 입출력을 종료하기 위해 닫음 |

# open 함수 매개변수 mode의 값



- open 함수
  - open은 파일을 읽고 쓰기 전에 해야하는 첫 과정
  - open 과정에서 파일이 존재하는지, 파일이 접근 허가가 있는지, 디스크의 저장 공간은 충분한지 검사됨
    - 이 검사가 실패하면 open이 실패하고 파일을 읽거나 쓰는 과정 중단
- open 함수 매개변수 mode의 값
  - 파일을 읽을 것인지, 생성할 것인지, 바이너리 데이터를 다룰 지 등을 지시

| 열기 모드 | 내용 |
|---|---|
| "r" | '읽기 모드'라고 부르며, 파일을 읽을 때 사용. mode 값이 생략되면 디폴트가 "r"이다. |
| "w" | '쓰기 모드'. 파일에 데이터를 쓸 때 사용. 이미 파일이 존재하면 쓰기 시작점이 맨 앞으로 설정되고 뒤에 데이터는 삭제됨. 파일이 존재하지 않는다면 새 파일을 만들고 쓰기 시작점을 맨 앞으로 설정 |
| "a" | '추가 쓰기 모드'. "w"와 달리 기존에 존재하는 파일이면 파일의 끝을 쓰기 시작점으로 설정하여 파일 끝에서부터 이어 쓰기 진행 |
| "x" | 파일이 없으면 파일을 생성하고 '쓰기 모드'로 열기. 파일이 있으면 오류 발생 |
| "b" | 오디오나 비디오와 같은 바이너리 파일의 입출력을 위한 '바이너리 모드' 설정 |
| "t" | 텍스트 파일의 입출력을 위한 '텍스트 모드' 설정. "b"나 "t"가 주어지지 않으면 디폴트가 "t"임 |

# 예제 4-21 파일 쓰고 읽기



myfile.txt 파일을 새로 생성하고 여러 줄의 문자열을 저장하라. 그리고 myfile.txt 파일을 다시 읽어 출력하는 프로그램을 작성하라. 그리고 cat 명령으로 myfile.txt 파일을 화면에 출력하여 제대로 저장되었는지 확인하라.

4-21.py

```
names = ["손흥민", "황희찬", "박항서", "유상철", "김남일"]
file = open("myfile.txt", "w") # 쓰기 모드로 myfile.txt 파일 열기. myfile.txt가 없으면 생성

for i in range(0, len(names)): # len(names)가 5이므로 5번 반복
    line = "%d번째 이름 %s₩n" % (i, names[i])
    file.write(line) # 파일 쓰기
file.close( ) # 파일 닫기. 다시 열기 전에는 쓸 수 없음

# 파일 열기
file = open("myfile.txt", "r") # 읽기 모드로 myfile.txt 파일 열기. myfile.txt 파일이 없으면 예외 발생
for line in file.readlines( ): # 파일에서 각 라인을 반복하여 line 변수에 읽음
    print(line, end="") # 읽은 라인을 화면에 출력

file.close( ) # 파일 닫기. 다시 열기 전에는 읽을 수 없음
```

```
pi@pi:~/ch04 $ python 4-21.py
0번째 이름 손흥민
1번째 이름 황희찬
2번째 이름 박항서
3번째 이름 유상철
4번째 이름 김남일
pi@pi:~/ch04 $
```